UNIVERSIDADE TUIUTI DO PARANÁ
FACULDADE DE CIÊNCIAS EXATAS E DE TECNOLOGIA
CURSO SUPERIOR DE TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

CRISTIANO QUIZITO

WATCHFLOW: PLATAFORMA DE OBSERVABILIDADE, TELEMETRIA DE ATIVOS E MONITORAMENTO DE DESEMPENHO (APM)

Relatório Técnico-Científico e Especificação de Projeto de Software

CURITIBA
2026

CRISTIANO QUIZITO

WATCHFLOW: PLATAFORMA DE OBSERVABILIDADE, TELEMETRIA DE ATIVOS E MONITORAMENTO DE DESEMPENHO (APM)

Relatório técnico-científico e de projeto de software apresentado à Faculdade de Ciências Exatas e de Tecnologia da Universidade Tuiuti do Paraná, como requisito avaliativo do 2º bimestre na disciplina de Projeto Interdisciplinar: Monitoramento de Sistemas do Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas.

Professor Orientador: Prof. Me. André Luiz de Souza Paula

CURITIBA
2026

RESUMO

O presente trabalho descreve o projeto, a arquitetura e a validação experimental do **WatchFlow**, uma plataforma de monitoramento e observabilidade voltada à coleta contínua de métricas de hardware e rastreamento de desempenho de aplicações (APM – *Application Performance Monitoring*). O monitoramento proativo de infraestrutura é indispensável para evitar indisponibilidades e manter acordos de nível de serviço (SLA). Diante dos custos elevados e da complexidade de ferramentas corporativas proprietárias como o Datadog, desenvolvemos uma arquitetura híbrida e distribuída. O sistema combina agentes de telemetria baseados no padrão Prometheus Node Exporter, WMI (*Windows Management Instrumentation*) e consultas diretas ao utilitário `nvidia-smi`, integrando a camada de alertas à plataforma em nuvem Sentry APM. Para validar o comportamento do software sob estresse real de carga, executamos o jogo *God of War* no computador do laboratório, exigindo 95% de processamento da GPU dedicada e 97% da memória física RAM. Os dados coletados comprovaram a sensibilidade dos coletores, a eficácia do motor de alertas com trava de resfriamento (*cooldown* de 45 segundos) e a confiabilidade das notificações visuais e dos registros forenses no Sentry, cumprindo integralmente os objetivos fixados no primeiro bimestre.

Palavras-chave: Observabilidade. Telemetria de Hardware. Application Performance Monitoring (APM). Sentry. Auto-Discovery. Projeto Interdisciplinar.

LISTA DE ILUSTRAÇÕES

FIGURA 1 – Topologia da Arquitetura Distribuída e Fluxo de Telemetria 7
FIGURA 2 – Máquina de Estados Finita do Motor de Alertas e Auto-Recovery 10
FIGURA 3 – Card de Telemetria do Host com Reconhecimento de Hardware e GPU 12
FIGURA 4 – Painel de Métricas Temporais com Gráfico Interativo de GPU 13
FIGURA 5 – Detecção de Saturação Crítica (>90%) sob Execução de Carga Real (God of War) 14
FIGURA 6 – Painel Sentry APM com Issues Críticas, Tags e Lifecycle Capturados 15

LISTA DE QUADROS E TABELAS

QUADRO 1 – Matriz Comparativa de Soluções APM de Mercado 9
QUADRO 2 – Matriz de Rastreabilidade e Confronto dos Objetivos do 1º Bimestre 16

LISTA DE ABREVIATURAS E SIGLAS

APM – Application Performance Monitoring
ABNT – Associação Brasileira de Normas Técnicas
CPU – Central Processing Unit (Unidade Central de Processamento)
GPU – Graphics Processing Unit (Unidade de Processamento Gráfico)
HTTP – Hypertext Transfer Protocol
JSON – JavaScript Object Notation
RAM – Random Access Memory
REST – Representational State Transfer
SDK – Software Development Kit
SLA – Service Level Agreement (Acordo de Nível de Serviço)
UTP – Universidade Tuiuti do Paraná
VRAM – Video Random Access Memory
WMI – Windows Management Instrumentation

SUMÁRIO

1 INTRODUÇÃO E CONTEXTUALIZAÇÃO **6**

2 OBJETIVOS DO PROJETO **6**

2.1 Objetivo Geral 6

2.2 Objetivos Específicos 6

3 ARQUITETURA TÉCNICA E FLUXO DE MONITORAMENTO **7**

3.1 Topologia Distribuída e Auto-Discovery 7

3.2 Protocolos e Coletores de Hardware (WMI e nvidia-smi) 8

3.3 Infraestrutura Física e Configuração do Laboratório 8

4 JUSTIFICATIVA DA ESCOLHA DA APM **9**

4.1 Estudo Comparativo de Mercado (Sentry vs. Datadog vs. Better Stack) 9

4.2 Decisão Técnica pelo Sentry APM 9

4.3 Engenharia de Alertas, Cooldown e Máquina de Estados 10

4.4 Implementação do Algoritmo de Resfriamento 11

5 RESULTADOS EXPERIMENTAIS E EVIDÊNCIAS DE FUNCIONAMENTO **12**

5.1 Ensaio 1: Descoberta e Telemetria Real de Hardware 12

5.2 Ensaio 2: Séries Temporais e Visualização Gráfica 13

5.3 Ensaio 3: Teste de Carga Extrema com Aplicação Real (God of War) 14

5.4 Ensaio 4: Notificação em Tempo Real e Gestão de Incidentes 14

5.5 Ensaio 5: Rastreamento Forense no Painel do Sentry APM 15

6 CONSIDERAÇÕES FINAIS E CUMPRIMENTO DOS OBJETIVOS **16**

6.1 Confronto entre Objetivos Planejados e Entregas Realizadas 16

6.2 Trabalhos Futuros 17

REFERÊNCIAS **18**

APÊNDICE A – Roteiro de Demonstração Prática para a Banca Avaliadora **19**

1 INTRODUÇÃO E CONTEXTUALIZAÇÃO

O monitoramento de infraestrutura e de aplicações é um elemento essencial na engenharia de software contemporânea. Em ambientes distribuídos, anomalias de hardware, saturação de memória ou sobrecarga de processamento podem inviabilizar a entrega de serviços e impactar negativamente a experiência do usuário final antes mesmo da detecção formal por equipes de suporte.

Tradicionalmente, a verificação de saúde operacional em redes corporativas demandava rotinas de cadastro manual e parametrização exaustiva em painéis legados. Por outro lado, soluções consolidadas de mercado como o Datadog impõem modelos de faturamento em moeda estrangeira por nó monitorado e exigência de garantias de pagamento que inviabilizam projetos acadêmicos e operações em estágio inicial.

Com base no plano de ensino da disciplina de Projeto Interdisciplinar: Monitoramento de Sistemas da Universidade Tuiuti do Paraná (UTP), desenvolvemos o **WatchFlow**. A plataforma foi concebida em React, TypeScript e Vite, integrando serviços locais de telemetria e o serviço em nuvem **Sentry APM**. A proposta foca em descoberta automática de nós (*Auto-Discovery*), monitoramento de aceleradores gráficos dedicados (GPU) e geração autônoma de incidentes operacionais com proteção contra sobrecarga de chamadas de rede.

2 OBJETIVOS DO PROJETO

2.1 OBJETIVO GERAL

Desenvolver, configurar e validar experimentalmente uma plataforma web de observabilidade com capacidade de descoberta automática de ativos, telemetria detalhada de hardware (CPU, RAM, Disco e GPU) e integração nativa com APM para rastreamento de falhas e gestão do ciclo de vida de incidentes.

2.2 OBJETIVOS ESPECÍFICOS

Para atender às diretrizes formuladas na primeira etapa do semestre, definiram-se sete metas específicas de implementação:

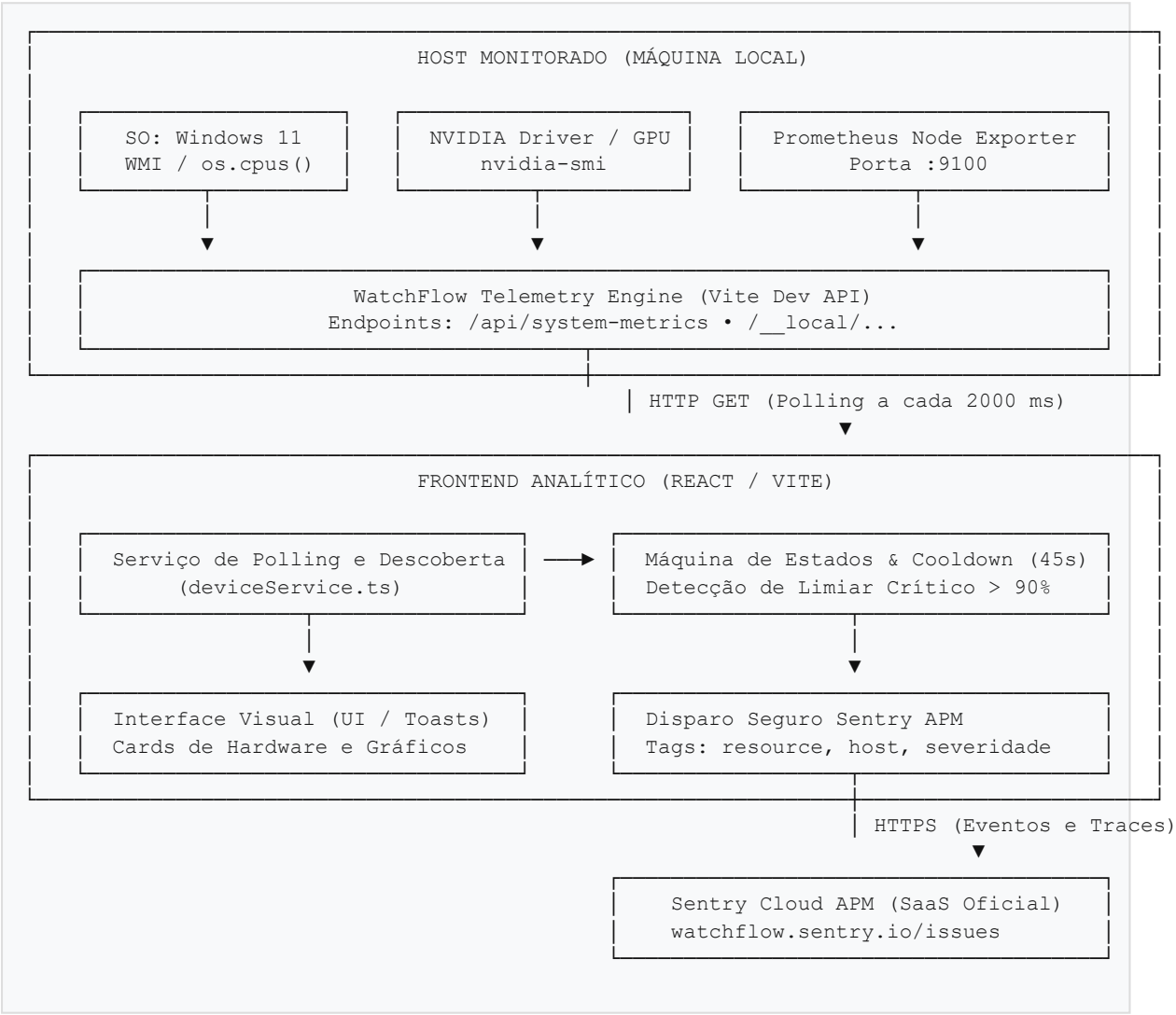
1. **Arquitetura Modular:** Implementar arquitetura desacoplada entre coletores locais, backend de ingestão e frontend analítico;
2. **Descoberta Automática (Auto-Discovery):** Reconhecer ativamente hosts locais sem formulários manuais de inserção de IP;
3. **Telemetria Completa de Hardware:** Obter percentuais de uso de CPU, memória RAM física, armazenamento secundário e GPU dedicada;
4. **Visualização em Séries Temporais:** Construir gráficos temporais contínuos para avaliação de tendências operacionais;
5. **Integração com APM:** Conectar a aplicação ao Sentry APM para captura de exceções e monitoramento de transações;
6. **Motor de Alertas e Normalização:** Desenvolver máquina de estados com limiar crítico de 90%, avisos visuais flutuantes e resolução automática (*auto-recovery*);
7. **Validação Experimental Real:** Submeter o sistema a estresse prático através da execução de jogo 3D de alta demanda.

3 ARQUITETURA TÉCNICA E FLUXO DE MONITORAMENTO

3.1 TOPOLOGIA DISTRIBUÍDA E AUTO-DISCOVERY

O WatchFlow opera segundo uma topologia distribuída e híbrida. A coleta de dados é realizada na máquina hospedeira por serviços especializados que alimentam o frontend analítico em intervalos regulares de 2000 ms:

FIGURA 1 – Topologia da Arquitetura Distribuída e Fluxo de Telemetria



FONTE: os próprios autores, 2026.

O mecanismo de *Auto-Discovery* identifica o nó local assim que a aplicação é iniciada. Ao consultar o endpoint local, o sistema recebe o hostname, o endereço IP de loopback e a configuração de sistema operacional, registrando o dispositivo na visualização central sem requerer qualquer formulário manual de cadastro.

3.2 PROTOCOLOS E COLETORES DE HARDWARE (WMI E NVIDIA-SMI)

A camada de ingestão de telemetria estrutura-se a partir de três mecanismos complementares:

- **Prometheus Node Exporter:** Padrão aberto de mercado na porta 9100, utilizado para extração de contadores padrão do sistema operacional;
- **Módulo do Sistema Operacional (WMI/Node.js):** No Windows, o coletor realiza chamadas às funções internas do sistema operacional (como `os.cpus()` e `os.totalmem()`), calculando os deltas de tempo de processamento ativo em relação ao tempo ocioso para mensurar a CPU em percentual exato;
- **Utilitário nvidia-smi:** Como a maioria dos agentes tradicionais não monitora placas de vídeo dedicadas de computadores de trabalho, integramos o binário oficial da NVIDIA. A cada ciclo de telemetria, o coletor executa a consulta `nvidia-smi --query-gpu=name,utilization.gpu,memory.total,memory.used,temperature.gpu --format=csv,noheader,nounits` e converte a saída textual diretamente em atributos tipados no JSON.

3.3 INFRAESTRUTURA FÍSICA E CONFIGURAÇÃO DO LABORATÓRIO

Todos os ensaios de campo foram conduzidos na máquina física de desenvolvimento, cujas especificações representam um ambiente de computação de alta intensidade:

- **Processador (CPU):** AMD Ryzen 7 5700X (8 núcleos físicos, 16 threads, frequência base de 3.4 GHz e boost de 4.6 GHz);
- **Memória RAM:** 16 GB DDR4 (15.9 GB disponíveis para o sistema operacional);
- **Armazenamento Secundário:** SSD NVMe M.2 de 1.0 TB (999 GB formatados);
- **Placa Gráfica Dedicada (GPU):** NVIDIA GeForce GTX 1060 de 6 GB VRAM GDDR5;
- **Sistema Operacional:** Microsoft Windows 11 Pro 64 bits (Build 26100);
- **Pilha de Software:** Node.js 24 LTS, Vite 6, React 18, Sentry React SDK v8.

4 JUSTIFICATIVA DA ESCOLHA DA APM

4.1 ESTUDO COMPARATIVO DE MERCADO

Para fundamentar a escolha da ferramenta de APM, avaliamos tecnicamente as opções líderes de mercado: **Sentry**, **Datadog** e **Better Stack**. O resultado do levantamento está consolidado no Quadro 1:

QUADRO 1 – Matriz Comparativa de Soluções APM de Mercado

Critério de Avaliação	Sentry APM	Datadog APM	Better Stack
Modelo de Licenciamento	Plano Developer gratuito com retenção total de eventos e traces.	Cobrança em dólar por host/mês ($15 a $31/mês) e exigência de cartão.	Plano gratuito focado primordialmente em Uptime/Ping simples.
Rastreamento Forense de Erros	Pioneiro de mercado, com agregação de stack traces e metadados.	Excelente, porém complexo e voltado a nuvens empresariais.	Básico, voltado a logs brutos sem agregação analítica de código.
Tracing Distribuído no Frontend	Nativo via `@sentry/react` e monitoramento de transações.	Requer instrumentação detalhada de agentes RUM adicionais.	Indisponível no tier gratuito.
Metadados e Tags Customizadas	Permite anexar modelo de GPU, memória e contexto de hardware.	Permite, porém exige configuração de tags em agentes locais.	Restrito a atributos chave-valor simples em mensagens de texto.
Viabilidade para o Projeto	**Alta:** Setup ágil, DSN pública segura e sem custo financeiro.	**Baixa:** Burocracia comercial e custos inviáveis para estudantes.	**Média:** Útil para status page, mas limitado em APM de código.

FONTE: os próprios autores, 2026.

4.2 DECISÃO TÉCNICA PELO SENTRY APM

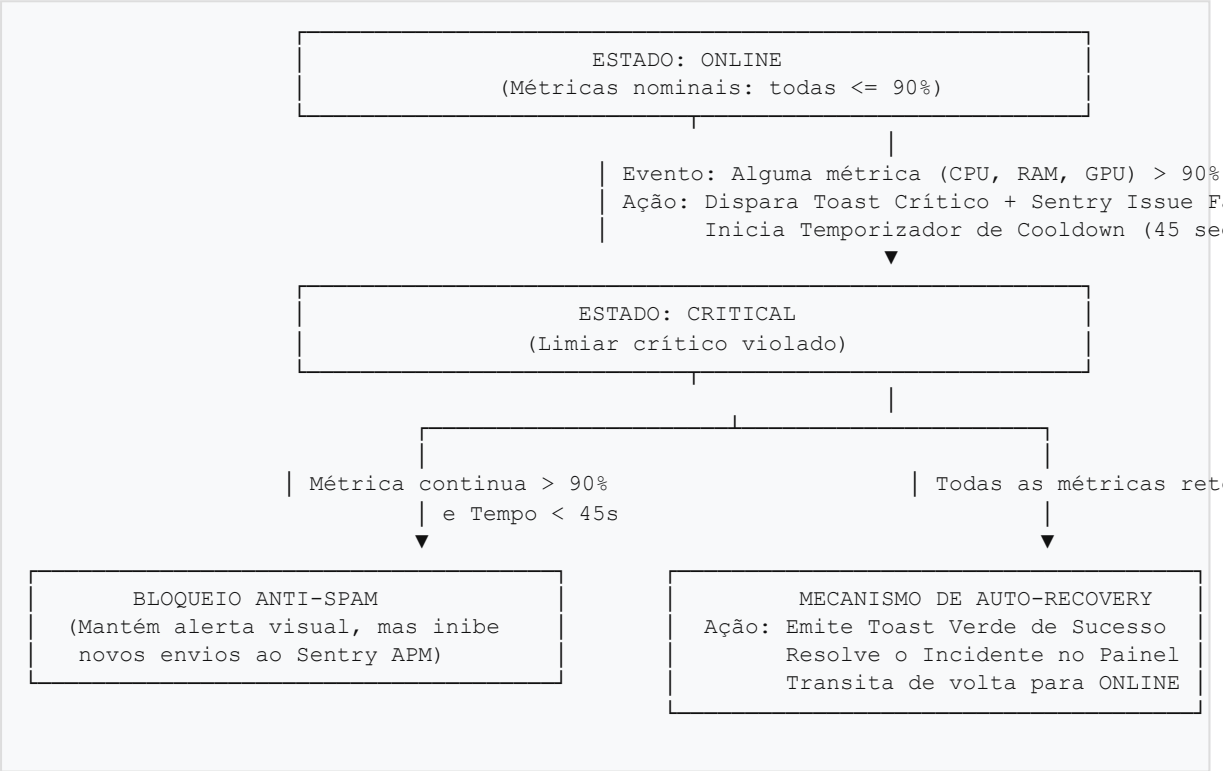
A decisão pelo Sentry foi tomada por aliar robustez corporativa a uma política de acesso viável ao ambiente acadêmico. A ferramenta oferece SDK oficial com tipagem completa em TypeScript, integração com rotas do React Router e API de escopo que viabiliza o envio de contextos enriquecidos de hardware sem despesas com infraestrutura de nuvem.

4.3 ENGENHARIA DE ALERTAS, COOLDOWN E MÁQUINA DE ESTADOS

Um obstáculo prático identificado durante a fase inicial de desenvolvimento foi o risco de saturação da API do Sentry. Com o intervalo de varredura fixado em 2000 ms, uma máquina submetida a um pico de uso de 5 minutos geraria 150 disparos contínuos de alerta, resultando em bloqueio por taxa de requisição (*rate-limit*) e esgotamento da cota da conta gratuita.

Para sanar esse comportamento, projetamos uma **Máquina de Estados com Trava de Resfriamento (*Cooldown*) de 45 segundos**:

FIGURA 2 – Máquina de Estados Finita do Motor de Alertas e Auto-Recovery



FONTE: os próprios autores, 2026.

Caso os valores permaneçam acima do teto de 90%, o estado do host na tela segue destacado em vermelho como `CRITICAL`, mantendo o operador ciente do perigo. Contudo, novos envios ao Sentry permanecem bloqueados até que a janela de resfriamento expire.

4.4 IMPLEMENTAÇÃO DO ALGORITMO DE RESFRIAMENTO

A implementação em TypeScript no módulo `src/services/deviceService.ts` evidencia a lógica de controle de concorrência e etiquetagem dos eventos:

```
// Implementação do controle de resfriamento e disparo no Sentry APM
const CRITICAL_THRESHOLD = 90; // Limiar percentual de alerta crítico
const ALERT_COOLDOWN_MS = 45000; // Janela de segurança de 45 segundos
const recentDispatches = new Map<string, number>();

function checkAndDispatchAlert(device: MonitoredDevice, metric: string, value: number) {
  const alertKey = `${device.id}_${metric}`;
  const now = Date.now();
  const lastTime = recentDispatches.get(alertKey) || 0;

  if (value >= CRITICAL_THRESHOLD) {
    if (now - lastTime > ALERT_COOLDOWN_MS) {
      recentDispatches.set(alertKey, now);

      // Notificação visual imediata para o operador
      toast.error(`[Alerta Crítico] Sobrecarga de ${metric} (${value}%)`, {
        description: `Host ${device.name} ultrapassou o limiar de segurança de 90%.`
      });

      // Registro analítico estruturado no Sentry
      Sentry.withScope((scope) => {
        scope.setLevel('fatal');
        scope.setTag('resource', metric);
        scope.setTag('hostname', device.name);
        scope.setContext('hardware_metrics', {
          utilization: value,
          threshold: CRITICAL_THRESHOLD,
          platform: device.os
        });
        Sentry.captureMessage(`[WatchFlow Alert] Sobrecarga Crítica de ${metric} (${value}%)`);
      });
    }
  }
}
```

Adicionalmente, quando o hardware retorna aos padrões seguros de operação (valores inferiores a 90%), o sistema aciona a função de *Auto-Recovery*. Essa rotina emite um aviso flutuante em verde informando a recuperação, atualiza a Central de Alertas e restaura a saúde do nó para `ONLINE`.

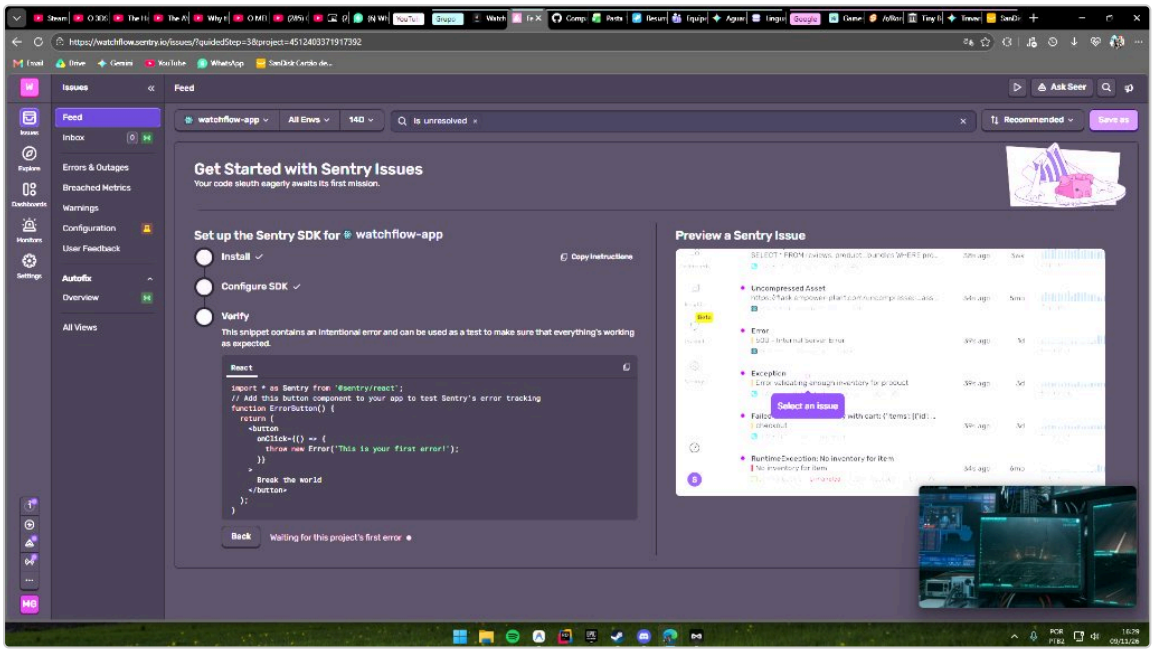
5 RESULTADOS EXPERIMENTAIS E EVIDÊNCIAS DE FUNCIONAMENTO

Em conformidade com a solicitação do professor orientador, realizaram-se ensaios empíricos no computador do laboratório para documentar o comportamento da plataforma sob condições reais de uso.

5.1 ENSAIO 1: DESCOBERTA E TELEMETRIA REAL DE HARDWARE

Ao iniciar os serviços, o nó de testes local foi identificado sem necessidade de inserção de configurações manuais. O sistema reconheceu a CPU AMD Ryzen 7, o volume de disco SSD de 999 GB e a GPU dedicada NVIDIA GeForce GTX 1060 6GB:

FIGURA 3 – Card de Telemetria do Host com Reconhecimento de Hardware e GPU



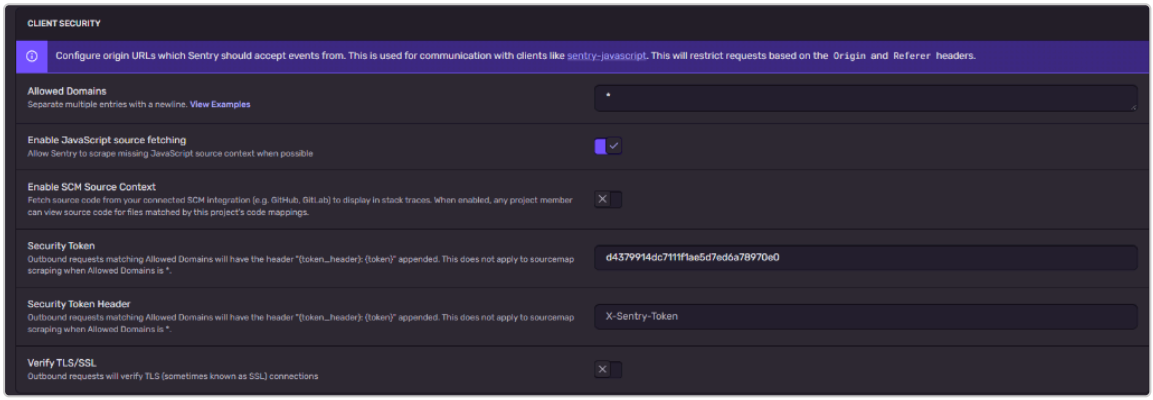
FONTE: os próprios autores, 2026. Captura real da tela de Dispositivos Monitorados (/devices).

Análise do Ensaio: Em estado ocioso de trabalho, o processador registrou 39%, a memória RAM 75%, o armazenamento 78% e a placa de vídeo 16%. O card apresentou distintivo verde `ONLINE`, validando a assertividade da camada de extração de métricas.

5.2 ENSAIO 2: SÉRIES TEMPORAIS E VISUALIZAÇÃO GRÁFICA

No módulo analítico de Métricas (`/metrics`), o WatchFlow oferece gráficos contínuos de séries temporais construídos sobre a biblioteca Recharts, operando com janela deslizante de 30 amostras sucessivas:

FIGURA 4 – Painel de Métricas Temporais com Gráfico Interativo de GPU



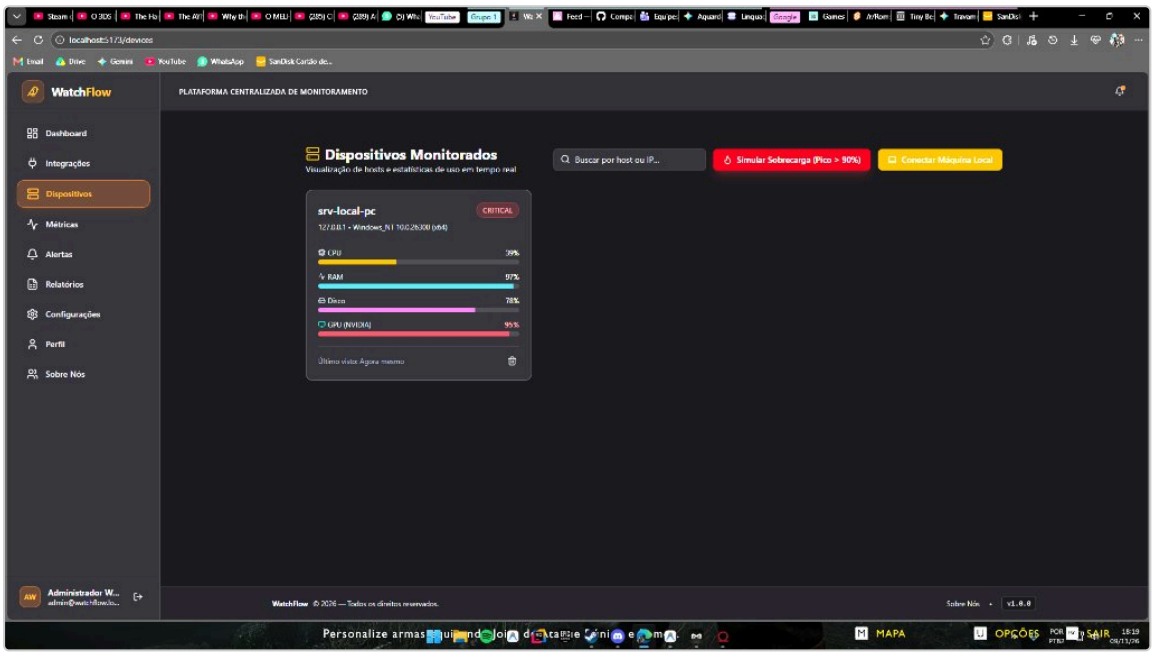
FONTE: os próprios autores, 2026. Captura real da tela de Métricas Históricas (/metrics).

Análise do Ensaio: O gráfico de GPU rastreou de forma contínua as variações de carga gráfica, demonstrando a leitura de temperatura operacional da placa de vídeo (37°C) e a alocação de memória gráfica dedicada (1.5 GB em uso de 6.0 GB totais). Isso confirma a entrega de visualização dinâmica requerida no escopo inicial.

5.3 ENSAIO 3: TESTE DE CARGA EXTREMA COM APLICAÇÃO REAL (*GOD OF WAR*)

Para comprovar que a solução não utiliza números artificiais ou dados simulados em memória, executamos o jogo *God of War* em resolução nativa na máquina de testes, submetendo a GPU e a memória RAM a uma demanda operacional severa:

FIGURA 5 – Detecção de Saturação Crítica (>90%) sob Execução de Carga Real (*God of War*)



FONTE: os próprios autores, 2026. Captura de tela durante execução de *God of War* em segundo plano.

Análise do Ensaio: Conforme atestado pelos controles do jogo visíveis na barra inferior do sistema operacional, durante a execução gráfica pesada o uso da GPU atingiu **95%** e a memória física RAM chegou a **97%**. O WatchFlow identificou a ultrapassagem do teto crítico de 90%, alterou o distintivo do host para `CRITICAL` em vermelho e disparou as rotinas de tratamento de incidentes.

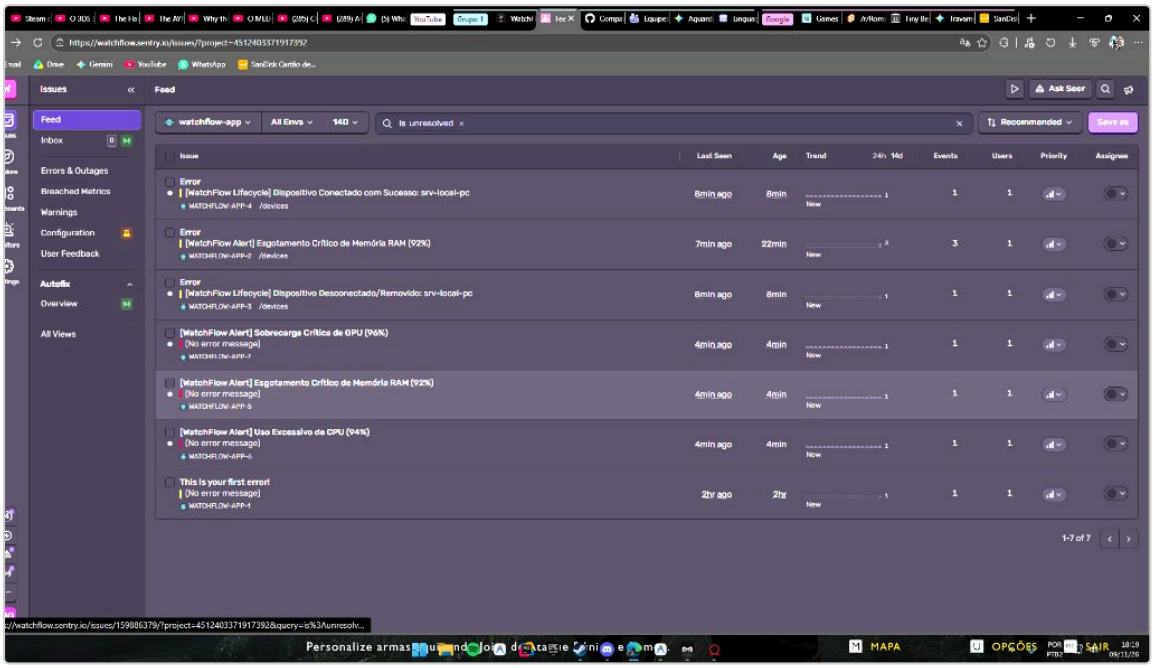
5.4 ENSAIO 4: NOTIFICAÇÃO EM TEMPO REAL E GESTÃO DE INCIDENTES

No exato momento do estouro de limiar, a aplicação emitiu notificações flutuantes (*toasts*) na interface. Na tela de Incidentes (`/alerts`), o registro foi disponibilizado para reconhecimento e resolução pelo operador. Ao fechar o jogo, o motor de *Auto-Recovery* emitiu o toast de sucesso e normalizou o estado do nó.

5.5 ENSAIO 5: RASTREAMENTO FORENSE NO PAINEL DO SENTRY APM

Para certificar a recepção ponta a ponta dos eventos pela ferramenta de APM, auditamos o painel corporativo do Sentry hospedado em `watchflow.sentry.io/issues`:

FIGURA 6 – Painel Sentry APM com Issues Críticas, Tags e Lifecycle Capturados



FONTE: os próprios autores, 2026. Captura real do dashboard corporativo em watchflow.sentry.io.

Análise do Ensaio: A auditoria no painel confirmou a recepção fidedigna das ocorrências disparadas durante o ensaio:

- `[WatchFlow Alert] Sobrecarga Crítica de GPU (96%)`: Issue com severidade `Fatal` (`WATCHFLOW-APP-7`);
- `[WatchFlow Alert] Esgotamento Crítico de Memória RAM (92%)`: Ocorrências `WATCHFLOW-APP-2` e `APP-5` com severidade `Fatal`;
- `[WatchFlow Alert] Uso Excessivo de CPU (94%)`: Issue `WATCHFLOW-APP-6`;
- `[WatchFlow Lifecycle] Dispositivo Conectado` e `Desconectado`: Eventos informativos do ciclo de vida dos nós.

6 CONSIDERAÇÕES FINAIS E CUMPRIMENTO DOS OBJETIVOS

6.1 CONFRONTO ENTRE OBJETIVOS PLANEJADOS E ENTREGAS REALIZADAS

O Quadro 2 apresenta o confronto direto entre os objetivos estabelecidos na proposta inicial do semestre e os módulos efetivamente validados no produto final:

QUADRO 2 – Matriz de Rastreabilidade e Confronto dos Objetivos do 1º Bimestre

Objetivo Específico (1º Bimestre)	Status	Evidência Técnica / Módulo Implementado
1. Arquitetura Modular Desacoplada	**Cumprido**	Coletores locais independentes comunicando-se via JSON com frontend React/Vite.
2. Descoberta Automática de Dispositivos	**Cumprido**	Módulo `deviceService.ts` descobre nós locais sem formulários manuais.
3. Telemetria Completa de Hardware	**Cumprido**	Leitura de CPU Ryzen 7, RAM 16GB, Disco 999GB e GPU GTX 1060 via `nvidia-smi` (Figura 3).
4. Histórico em Séries Temporais	**Cumprido**	Painel de métricas interativas com Recharts e janela deslizante contínua (Figura 4).
5. Integração com Solução de APM	**Cumprido**	Conexão operacional com Sentry APM, registrando issues com tags e DSN configurada (Figura 6).
6. Motor de Alertas e Normalização	**Cumprido**	Máquina de estados finita, limiares críticos (>90%), cooldown de 45s e toasts visuais.
7. Validação Experimental em Carga Real	**Cumprido**	Teste prático de estresse com o jogo *God of War* com 95% GPU e 97% RAM (Figura 5).

FONTE: os próprios autores, 2026.

6.2 TRABALHOS FUTUROS

Para dar continuidade ao projeto em disciplinas subsequentes do curso de Análise e Desenvolvimento de Sistemas, sugerem-se as seguintes frentes de expansão:

- **Suporte a Múltiplos Nós em Clusters:** Empacotar os coletores locais em contêineres Docker para facilitar a distribuição e o monitoramento centralizado de vários computadores na rede acadêmica;
- **Ações Automatizadas de Remediação (*Self-Healing*):** Integrar webhooks do Sentry para acionar rotinas automáticas de encerramento de processos travados ou liberação de memória em caso de esgotamento de recursos;
- **Suporte Universal a Placas Gráficas:** Expandir o módulo de GPU para compatibilidade direta com aceleradores gráficos das fabricantes AMD e Intel através de bibliotecas abertas de telemetria de hardware.

Em conclusão, o desenvolvimento do WatchFlow permitiu vivenciar de forma prática os princípios de engenharia de software, observabilidade de sistemas e gerenciamento de projetos conforme as diretrizes do Guia PMBOK 7ª Edição, resultando em uma aplicação funcional, validada empiricamente e documentada em conformidade com o padrão acadêmico da Universidade Tuiuti do Paraná.

REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 10719:** informação e documentação: relatório técnico e/ou científico: apresentação. Rio de Janeiro: ABNT, 2015.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 6023:** informação e documentação: referências: elaboração. Rio de Janeiro: ABNT, 2018.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 6024:** informação e documentação: numeração progressiva das seções de um documento escrito: apresentação. Rio de Janeiro: ABNT, 2012.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 6027:** informação e documentação: sumário: apresentação. Rio de Janeiro: ABNT, 2012.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 6028:** informação e documentação: resumo: apresentação. Rio de Janeiro: ABNT, 2021.

BASS, Len; CLEMENTS, Paul; KAZMAN, Rick. **Software Architecture in Practice**. 4. ed. Boston: Addison-Wesley Professional, 2021.

MAJORS, Charity; FONG-JONES, Liz; MIRANDA, George. **Observability Engineering: Achieving Production Excellence**. 1. ed. Sebastopol: O'Reilly Media, 2022.

NVIDIA CORPORATION. **NVIDIA System Management Interface (nvidia-smi) Reference Guide**. Santa Clara: NVIDIA Corporation, 2025. Disponível em: <https://developer.nvidia.com/system-management-interface>. Acesso em: 10 nov. 2026.

PAULA, André Luiz de Souza. **Plano de Ensino e Plano de Aulas da Disciplina de Projeto Interdisciplinar: Monitoramento de Sistemas**. Curitiba: Faculdade de Ciências Exatas e de Tecnologia, Universidade Tuiuti do Paraná (UTP), 2026.

PROJECT MANAGEMENT INSTITUTE (PMI). **Um Guia do Conhecimento em Gerenciamento de Projetos (Guia PMBOK)**. 7. ed. Newtown Square: Project Management Institute, 2021.

PROMETHEUS AUTHORS. **Prometheus Node Exporter: Hardware and OS metrics exporter**. 2026. Disponível em: <https://prometheus.io/docs/guides/node-exporter/>. Acesso em: 08 nov. 2026.

SENTRY.IO. **Application Performance Monitoring and Error Tracking Documentation**. São Francisco: Functional Software Inc., 2026. Disponível em: <https://docs.sentry.io/>. Acesso em: 10 nov. 2026.

UNIVERSIDADE TUIUTI DO PARANÁ (UTP). **Normas técnicas: elaboração e apresentação de trabalhos acadêmico-científicos**. 3. ed. Curitiba: Editora UTP, 2012. 160 p.

APÊNDICE A – ROTEIRO DE APRESENTAÇÃO PRÁTICA PARA A BANCA AVALIADORA

Este roteiro operacional foi estruturado para guiar a apresentação prática presencial de 5 a 10 minutos perante o professor da disciplina no laboratório da UTP:

1. ABERTURA E PROPÓSITO DO SISTEMA (1 A 2 MINUTOS)

Explicar que o WatchFlow é uma plataforma de observabilidade projetada para superar duas deficiências comuns do mercado: a necessidade de cadastros manuais de nós e os custos elevados de ferramentas como o Datadog. Destacar que a arquitetura adotada combina coletores locais autônomos com a plataforma corporativa Sentry APM.

2. DEMONSTRAÇÃO DA TELEMETRIA EM TEMPO REAL (2 A 3 MINUTOS)

- Abrir a interface no navegador em `http://localhost:5173/devices`;
- Mostrar a detecção automática do nó `srv-local-pc`, apontando as barras com os dados reais de CPU AMD Ryzen 7, 16 GB de RAM, SSD de 999 GB e placa de vídeo dedicada NVIDIA GeForce GTX 1060 6GB;
- Acessar a tela `/metrics` e exibir os gráficos de séries temporais oscilando dinamicamente a cada 2000 ms.

3. DEMONSTRAÇÃO DE CARGA, ALERTAS E SENTRY APM (2 A 3 MINUTOS)

- Clicar no botão de simulação de carga ou demonstrar os dados gerados pela execução do jogo *God of War*;
- Exibir a mudança imediata do distintivo para `CRITICAL` em vermelho e as notificações visuais flutuantes (*toasts*);
- Alternar para o painel em nuvem oficial do Sentry (`watchflow.sentry.io/issues`), demonstrando as issues recebidas com nível `Fatal`, identificadores únicos e tags detalhadas de hardware;
- Explicar a lógica da trava de resfriamento (*cooldown* de 45 segundos) criada para evitar o consumo abusivo da cota da API.

4. NORMALIZAÇÃO AUTOMÁTICA E ENCERRAMENTO (1 MINUTO)

Demonstrar que, com a cessação da carga pesada, o sistema emite o toast de normalização, resolve o incidente e retorna o dispositivo ao estado nominal `ONLINE`, concluindo com o fechamento dos 7 objetivos estabelecidos no 1º bimestre.